Mentimeter

Racket: Funcții ca valori. Funcționale

Mentimeter

Functii ca valori de prim rang

pot fi manipulate ca orice alta valoare

Mentimeter

Exemple

- legare la un identificator: (define f list?)
- pasate ca argumente: (map list '(1 2 3)), (list cons 1 'a)

Mentimeter

Functii uncurry

- primeste toti parametrii deodata
- ex: (define add-uncurry (lambda (x y)(+ x y)))

Mentimeter

Functii curry

- intoarce o noua functie atunci cand este aplicata pe mai putini parametri
- ex: (define add-curry (lambda (x) (lambda (y) (+ x y))))

Mentimeter

Transformare curry -> uncurry

- ex: (define add-curry (lambda (x) (lambda (y) (+ x y))))
- (define converted-add-curry (lambda (x y) ((add-curry x) y)))
- trebuie sa ne definim o noua functie in care sa pasam parametri pe rand

Mentimeter

Transformare uncurry -> curry

- ex: (define add-uncurry (lambda (x y) (+ x y)))
- (define converted-add-uncurry (lambda (x) (lambda (y) (add-uncurry x y))))
- **trebuie sa ne definim o noua functie in care sa primim parametri pe rand**

Mentimeter

Functionale

functii care manipuleaza alte functii (primite ca argumente / returnate din functie)

Mentimeter

compose f g

- compunerea functiilor f si g
- ((compose f g) x) = (f (g x))

Mentimeter

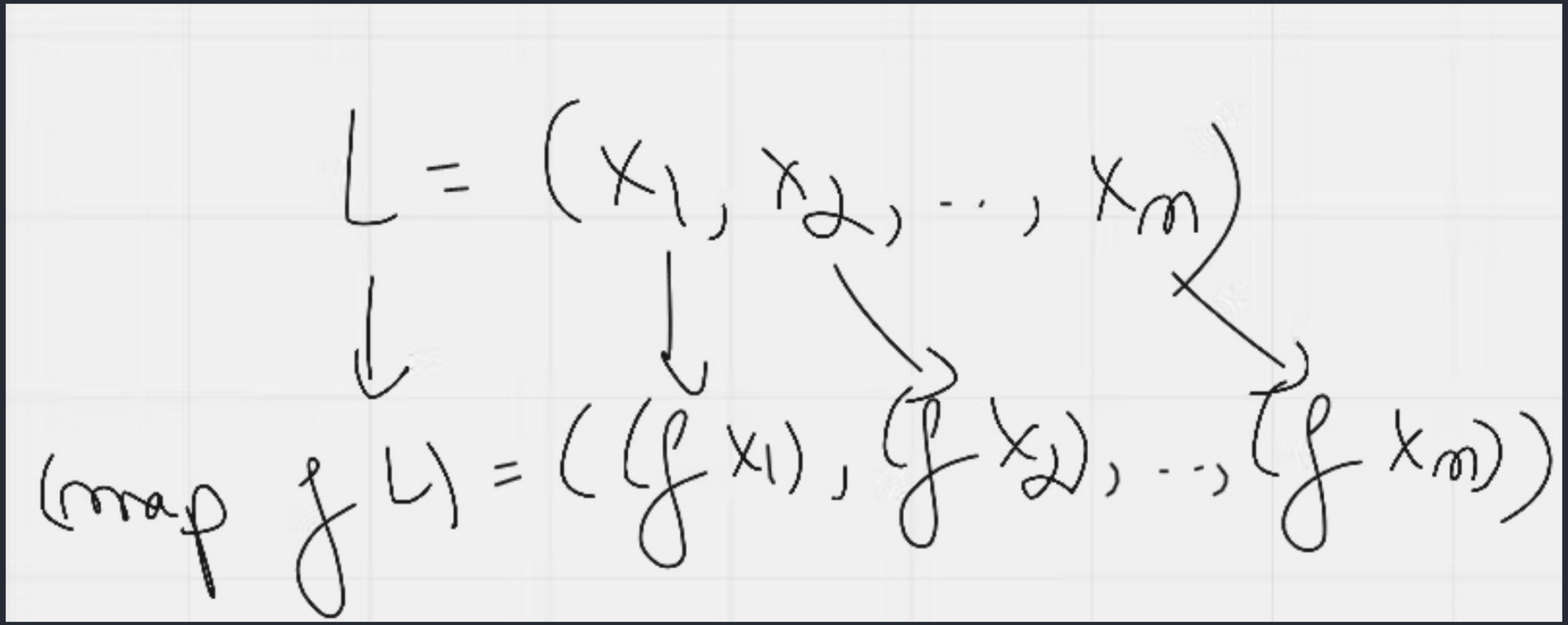
Exemple compose

- ((compose positive? -) 1) // (positive? (- 1)) = #f
- ((compose *) 1 2) // (composite f) = f
- ((compose + *) 2 3 4) // (+ (* 2 3 4)) = (+ 24) = 24

Mentimeter

map f L1 L2 ...

- cel putin o lista, dar pot fi oricate
- f primeste un numar de parametri egal cu numarul de liste primite
- aplica functia f pe fiecare element si intoarce noua lista


L = (x1, x2, . . , xm)
(map f L) = ((f x1), (f x2), . . , (f xm))

Mentimeter

Exemple map

- (map (lambda(x) (* x x)) '(1 2 3)) // '(1 4 9)
- (map + '(1 2 3) '(4 5 6) '(7 8 9)) // '(12 15 18)
- (map zero? (list 1 0 0 0 1)) // '(#f #t #t #t #f)

Mentimeter

filter f L

- o singura lista L si o functie f cu un singur parametru
- returneaza elementele din lista pentru care f intoarce true

Mentimeter

$$L = (x_1, x_2, \ldots, x_n)$$

$$\downarrow$$

(filter f L) = (... xi ...) dacă (f xi) = #t

Mentimeter

Exemple filter

- (filter positive? '(1 -10 100 2 -3)) // '(1 100 2)
- (filter (lambda (x) x) '(1 -10 #f)) // nu este eroare chiar daca functia nu intoarce mereu bool (rezultat '(1 -10))

Mentimeter

foldl/foldr f init l1 l2 ...

- cel putin o lista, dar pot fi oricate
- f primeste un numar de parametri egal cu numarul de liste primite + 1 (acumulatorul)
- returnează rezultatul aplicării lui f pe rând asupra unui element din listă și a acumulatorului

Mentimeter

foldl vs foldr

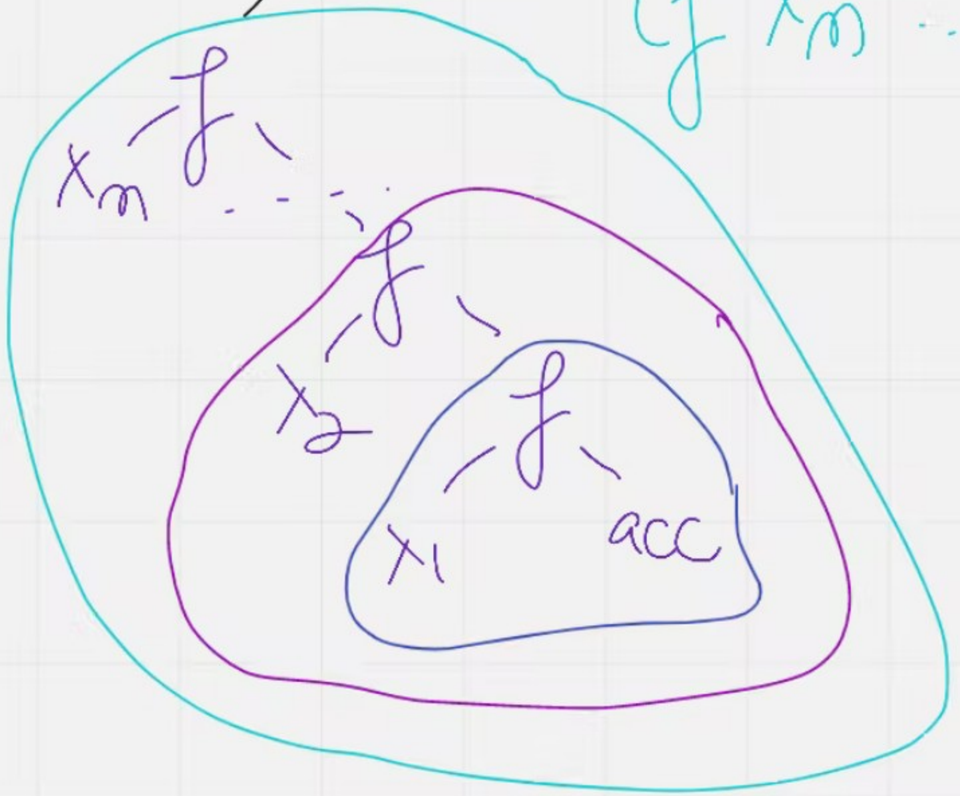
- foldl: elementele sunt parcurse de la stanga la dreapta
- foldr: elementele sunt parcurse de la dreapta la stanga

Mentimeter

$L = (x_1, x_2, \ldots, x_n)$

$\downarrow$

$(\text{foldl } f \text{ acc } L) =$



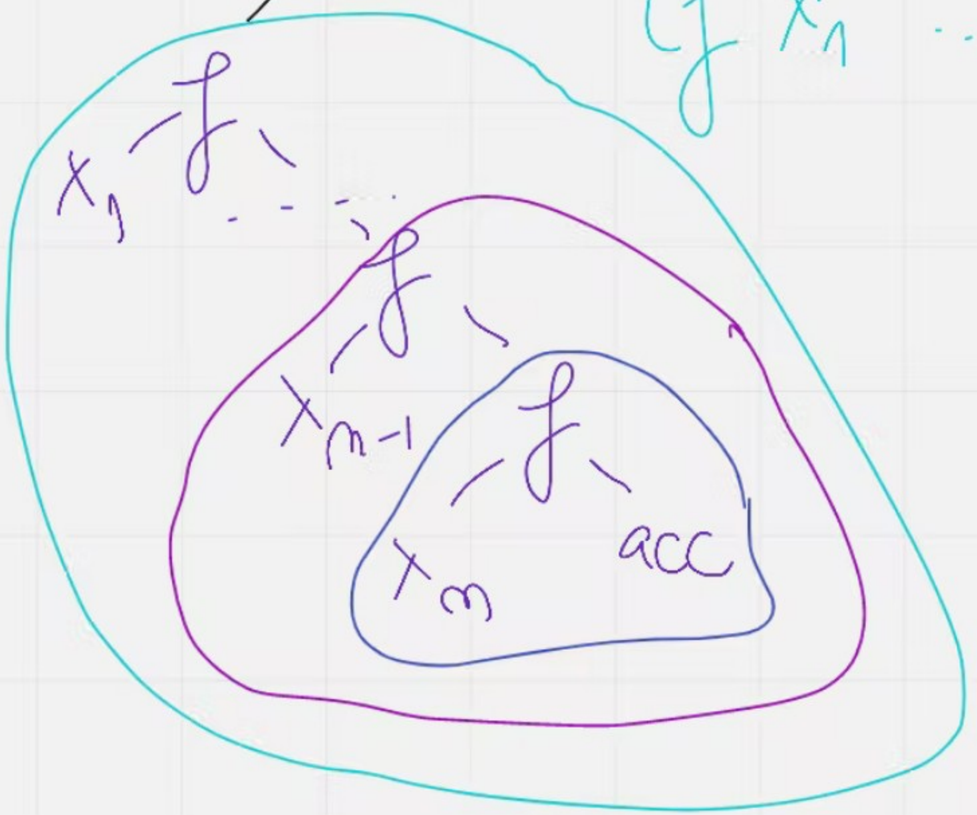
$(f\ x_n \ldots (f\ x_2\ (f\ x_1\ acc)) \ldots)$

Mentimeter

$L = (x_1, x_2, \ldots, x_n)$

$\downarrow$

(foldr f acc L) =



$(f\ x_1\ \ldots\ (f\ x_{m-1}\ (f\ x_m\ acc))\ldots)$

Mentimeter

foldl vs foldr

- foldl: recursivitate pe coada
- foldr: recursivitate pe stiva

Mentimeter

Exemple foldl/foldr

- (foldl append null '((1) (2)) '((3) (4))) // '(2 4 1 3)
- (foldr append null '((1) (2)) '((3) (4))) // '(1 3 2 4)
- (foldl (lambda (d res) (+ (* res 10) d)) 0 '(1 2 3)) // 123
- (foldr (lambda (d res) (+ (* res 10) d)) 0 '(1 2 3)) // 321

Mentimeter

apply f ... L

- primeste o functie, o lista si oricate argumente intre
- aplica f pe acele argumente + toate elementele din lista

Mentimeter

$$L = (x_1, x_2, \ldots, x_m)$$

$$\downarrow$$

$$(\text{map } f\ y_1 \ldots y_m\ L) = (f\ y_1 \ldots y_m\ x_1 \ldots x_m)$$

Mentimeter

Exemple apply

- (apply * 1 10 '(2 3 4)) // 240
- (apply map + '((1 2) (3 4) (5 6))) // (map + '(1 2) '(3 4) '(5 6)) = '(9 12)

Mentimeter

Exemplu mai complicat apply

- (apply foldl (lambda (f x res) (cons (f x) res)) null (list (list add1 sub1) '(2 5)))
- = (foldl (lambda (f x res) (cons (f x) res)) null (list add1 sub1) '(2 5))
- = (cons (sub1 5) (cons (add1 2) null)) = '(4 3)